



TOMATODOC: AN AI-POWERED TOMATO DISEASE MANAGEMENT PLATFORM

# **Observation-Based Tomato Disease Severity and Recovery Monitoring**

(A COMPONENT OF AN AI-POWERED TOMATO DISEASE MANAGEMENT PLATFORM)

## **INDIVIDUAL THESIS**

**Vithusha Pathmanathan**

IT22208262

Individual component: Observation-Based Tomato Disease Severity and Recovery Monitoring

**Group ID: RP-R26-SE-032**

Report submitted in partial fulfilment of the requirements for the

B.Sc. (Hons) in Information Technology

Specializing in Software Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

## DECLARATION AND SUPERVISOR CERTIFICATION

**Candidate declaration.** I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

**Academic integrity and AI disclosure.** I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Vithusha Pathmanathan

Student ID: IT22208262

| Name                  | Student ID | Signature   |
|-----------------------|------------|-------------|
| Pathmanathan Vithusha | IT22208262 | P. Vithusha |

**Supervisor certification.** The above candidate carried out this research for the bachelor's degree thesis under my supervision.

| Role and name                   | Signature                                                                           | Date       |
|---------------------------------|-------------------------------------------------------------------------------------|------------|
| Supervisor:<br>Dinithi Gunapada |  | 09.10.2026 |

## ABSTRACT

Tomato cultivation is economically important for smallholder farmers in Sri Lanka, yet disease management is often limited to one-time visual identification. After a disease is identified, farmers still need a practical way to tell whether severity is improving, stable, or worsening across repeated observations of the same plant. Manual monitoring is subjective, inconsistent, and difficult to track over time.

This thesis presents the Observation-Based Disease Severity and Recovery Monitoring component of the TomatoDoc platform for tomato leaf and fruit. The system accepts repeated uploads (Observation 1, 2 and 3) and validates each image with a hybrid OpenCV and MobileNetV2 gate. Relative severity is estimated mainly from the OpenCV affected-area percentage with a 30% LOW/HIGH threshold, and EfficientNet-B0 embeddings with cosine similarity give visual consistency evidence between observations. Weather context, trend states (BASELINE, IMPROVING, WORSENING, STABLE, RECOVERED), a monitoring insight, and a PDF report support farmer decisions without prescribing treatment. The backend uses FastAPI and SQLite, and the mobile client uses Expo React Native.

On the holdout evaluation the leaf gate passed 500/500 valid images and falsely accepted 0/225 reject samples. The leaf EfficientNet-B0 supporting model achieved 81.42% test accuracy and 81.60% F1 on pseudo-labelled severity data. Automated and manual tests covered the observation pipeline. The individual contribution is a transparent, repeatable severity and recovery monitoring workflow that stays separate from disease identification and treatment.

**Keywords:** tomato disease monitoring, OpenCV affected-area severity, MobileNetV2 gate, EfficientNet-B0 embedding, recovery trend.

## **ACKNOWLEDGEMENTS**

I would like to express my sincere gratitude to my supervisor and co-supervisor for their continuous guidance, valuable suggestions, and constructive feedback throughout this research project.

I also thank the lecturers, instructors, and the academic and non-academic staff of the Sri Lanka Institute of Information Technology for the learning environment and resources that made this work possible.

My sincere appreciation goes to my group members for their cooperation and teamwork across the TomatoDoc components. Their collaboration enabled clear module boundaries between disease identification, nutrient analysis, and disease severity monitoring.

Finally, I thank my parents, family members, and friends for their constant encouragement and support.

# TABLE OF CONTENTS

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>1 INTRODUCTION.....</b>                                    | <b>10</b> |
| 1.1 Overall project context.....                              | 10        |
| 1.2 Individual component context.....                         | 10        |
| 1.3 Problem statement.....                                    | 11        |
| 1.4 Research gap .....                                        | 11        |
| 1.5 Aim and objectives .....                                  | 11        |
| 1.6 Research questions or hypotheses.....                     | 12        |
| 1.7 Scope limitations and assumptions .....                   | 12        |
| 1.8 Individual contribution.....                              | 13        |
| 1.9 Thesis structure .....                                    | 13        |
| <b>2 LITERATURE REVIEW AND RELATED WORK .....</b>             | <b>14</b> |
| 2.1 Review method and scope .....                             | 14        |
| 2.2 Domain and theoretical foundations .....                  | 14        |
| 2.3 Related research .....                                    | 14        |
| 2.4 Existing systems products and techniques.....             | 14        |
| 2.5 Gap synthesis and conceptual framework.....               | 15        |
| <b>3 RESEARCH METHODOLOGY .....</b>                           | <b>16</b> |
| 3.1 Research design .....                                     | 16        |
| 3.2 Data participants or study materials.....                 | 17        |
| 3.3 Experimental or study procedure .....                     | 18        |
| 3.4 Measures and evaluation metrics .....                     | 19        |
| 3.5 Data analysis .....                                       | 19        |
| 3.6 Ethics privacy security and responsible AI.....           | 20        |
| 3.7 Reproducibility and research artefacts.....               | 20        |
| <b>4 PRODUCT AND SOLUTION DESIGN.....</b>                     | <b>21</b> |
| 4.1 Requirements .....                                        | 21        |
| 4.2 Component architecture within the complete solution ..... | 22        |
| 4.3 Design alternatives and decisions .....                   | 23        |
| 4.4 Detailed design.....                                      | 24        |
| 4.5 Standards and professional practice.....                  | 25        |
| <b>5 IMPLEMENTATION INTEGRATION AND TESTING .....</b>         | <b>26</b> |
| 5.1 Development environment and technology stack .....        | 26        |
| 5.2 Component implementation.....                             | 26        |
| 5.3 Integration with the other components .....               | 26        |
| 5.4 Verification and software testing.....                    | 27        |
| 5.5 Deployment and operation.....                             | 27        |
| <b>6 RESULTS AND VALIDATION.....</b>                          | <b>28</b> |
| 6.1 Research results .....                                    | 28        |
| 6.2 Product and technical results .....                       | 29        |

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| 6.3 User or stakeholder validation .....                              | 30        |
| 6.4 Integration results.....                                          | 30        |
| 6.5 Summary of findings.....                                          | 30        |
| <b>7 DISCUSSION .....</b>                                             | <b>31</b> |
| 7.1 Interpretation of findings .....                                  | 31        |
| 7.2 Comparison with literature baselines and existing solutions ..... | 31        |
| 7.3 Research contribution .....                                       | 31        |
| 7.4 Product contribution.....                                         | 31        |
| 7.5 Limitations and threats to validity .....                         | 31        |
| 7.6 Implications.....                                                 | 32        |
| <b>8 COMMERCIALISATION AND SUSTAINABILITY.....</b>                    | <b>33</b> |
| 8.1 Value proposition and target users.....                           | 33        |
| 8.2 Market and competitive position .....                             | 33        |
| 8.3 Business or sustainability model.....                             | 33        |
| 8.4 Scalability intellectual property and compliance.....             | 33        |
| 8.5 Contribution to the Sustainable Development Goals.....            | 33        |
| <b>9 CONCLUSION AND FUTURE DIRECTIONS .....</b>                       | <b>34</b> |
| 9.1 Conclusion .....                                                  | 34        |
| 9.2 Achievement of objectives.....                                    | 34        |
| 9.3 Individual research and product contributions.....                | 34        |
| 9.4 Future directions .....                                           | 34        |
| <b>REFERENCES.....</b>                                                | <b>36</b> |
| <b>GLOSSARY.....</b>                                                  | <b>36</b> |
| <b>APPENDICES .....</b>                                               | <b>38</b> |
| Appendix A Research instruments or extended results .....             | 38        |
| Appendix B Academic integrity and AI use disclosure.....              | 40        |

## LIST OF FIGURES

|                                                                                            |    |
|--------------------------------------------------------------------------------------------|----|
| Figure 1.1: Tomato disease monitoring context and farmer need.....                         | 10 |
| Figure 1.2: High-level TomatoDoc component boundary (Disease Monitoring).....              | 11 |
| Figure 3.1: Agile development lifecycle used for the component .....                       | 16 |
| Figure 3.2: Gantt chart for component milestones.....                                      | 17 |
| Figure 3.3: Work breakdown structure of the Disease Monitoring component.....              | 17 |
| Figure 3.4: Class distribution of the pseudo-labelled severity dataset.....                | 18 |
| Figure 4.1: System architecture of observation-based monitoring .....                      | 23 |
| Figure 4.2: Hybrid image gate pipeline (OpenCV and MobileNetV2).....                       | 24 |
| Figure 4.3: OpenCV affected-area severity estimation.....                                  | 24 |
| Figure 4.4: EfficientNet-B0 embedding extraction for visual consistency .....              | 25 |
| Figure 4.5: Trend analysis decision logic .....                                            | 25 |
| Figure 5.1: Mobile observation workflow (Create Case, Upload, Result, Overall Report)..... | 26 |
| Figure 6.1: Leaf gate holdout results .....                                                | 28 |
| Figure 6.2: Leaf EfficientNet-B0 supporting model metrics .....                            | 29 |
| Figure 6.3: Structure of the overall monitoring summary (Initial / Peak / Final) .....     | 29 |
| Figure 6.4: Visual consistency thresholds used across Observation 1 to 3.....              | 29 |

## LIST OF TABLES

|                                                                |    |
|----------------------------------------------------------------|----|
| Table 2.1: Research gap summary .....                          | 15 |
| Table 3.1: Datasets and study materials.....                   | 17 |
| Table 3.2: Models used in the component .....                  | 19 |
| Table 3.3: Consistency thresholds .....                        | 19 |
| Table 4.1: Use case: tomato farmer monitoring flow .....       | 21 |
| Table 4.2: Use case scenario UC-01 .....                       | 22 |
| Table 4.3: Main design decisions.....                          | 23 |
| Table 5.1: Development environment and tools .....             | 26 |
| Table 5.2: Test case summary for the observation pipeline..... | 27 |
| Table 6.1: Leaf gate holdout summary .....                     | 28 |
| Table 6.2: Leaf EfficientNet supporting model metrics.....     | 28 |
| Table 6.3: Summary of findings .....                           | 30 |
| Table 8.1: Budget plan (indicative).....                       | 33 |
| Table 9.1: Achievement of objectives.....                      | 34 |
| Table A.1: Test case TC01.....                                 | 38 |
| Table A.2: Test case TC02.....                                 | 38 |
| Table A.3: Test case TC03.....                                 | 38 |
| Table A.4: Test case TC04.....                                 | 38 |
| Table A.5: Test case TC05.....                                 | 39 |
| Table A.6: Test case TC06.....                                 | 39 |
| Table A.7: Test case TC07.....                                 | 39 |
| Table A.8: Test case TC08.....                                 | 40 |
| Table A.9: Test case TC09.....                                 | 40 |
| Table A.10: Test case TC10.....                                | 40 |

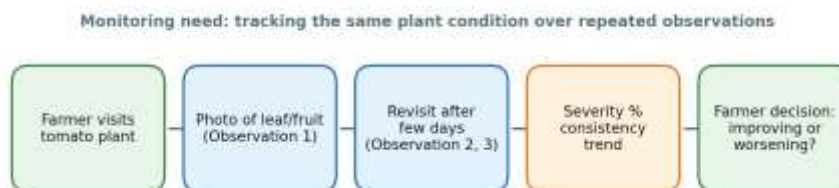
## LIST OF ABBREVIATIONS

| Abbreviation | Meaning                             |
|--------------|-------------------------------------|
| AI           | Artificial Intelligence             |
| API          | Application Programming Interface   |
| CNN          | Convolutional Neural Network        |
| CV           | Computer Vision                     |
| DB           | Database                            |
| GPS          | Global Positioning System           |
| HSV          | Hue, Saturation, Value              |
| HTTP         | Hypertext Transfer Protocol         |
| JSON         | JavaScript Object Notation          |
| ML           | Machine Learning                    |
| OpenCV       | Open Source Computer Vision Library |
| PDF          | Portable Document Format            |
| REST         | Representational State Transfer     |
| RGB          | Red, Green, Blue                    |
| SDG          | Sustainable Development Goal        |
| UI           | User Interface                      |
| WBS          | Work Breakdown Structure            |
| YOLO         | You Only Look Once                  |

# 1 INTRODUCTION

## 1.1 Overall project context

Agriculture remains a critical sector for food security and rural livelihoods in Sri Lanka. Tomato is widely cultivated and is highly susceptible to foliar and fruit diseases such as early blight, late blight, leaf miner damage, anthracnose, blossom end rot, and spotted wilt virus. TomatoDoc is a multi-component, AI-powered tomato disease management platform developed as a group research project. Its components cover disease identification, nutrient analysis, and disease severity and recovery monitoring, and they communicate through agreed API contracts so that each member can work on a clearly bounded problem. Figure 1.1 illustrates the practical need that this component addresses: a farmer who revisits the same plant and wants to know how its condition is changing.



*Figure 1.1: Tomato disease monitoring context and farmer need*

## 1.2 Individual component context

This thesis reports the individual component Observation-Based Tomato Disease Severity and Recovery Monitoring. Early recognition of a disease type is useful, but it does not answer the next practical question a farmer has: whether the condition of the same plant is getting better or worse after a few days. In real farm conditions farmers inspect plants visually and rely on memory or informal notes, which is subjective and hard to compare across days.

The component is designed for tomato leaf and fruit monitoring cases. It validates uploaded images, estimates relative severity, checks visual consistency between successive photos, attaches optional weather context, and reports a trend status to the farmer. Disease identification is deliberately treated as an external contract so that this module stays specialised for monitoring rather than diagnosis. Figure 1.2 shows where the component sits in the platform.

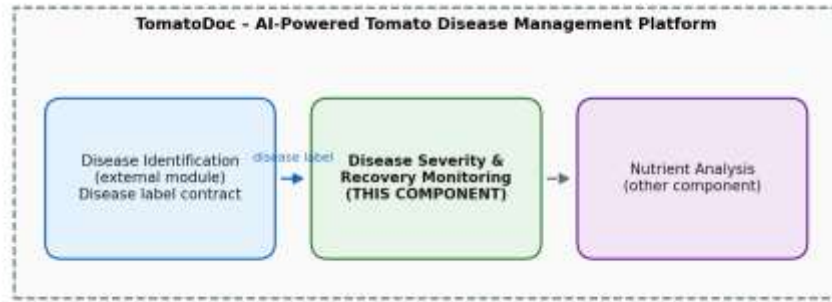


Figure 1.2: High-level TomatoDoc component boundary (Disease Monitoring)

### 1.3 Problem statement

Tomato farmers need more than a disease name. After identification they still find it difficult to compare plant condition objectively across days, to confirm that successive photos belong to the same monitoring sequence, and to tell whether severity is improving or worsening. The main difficulties are:

- subjective severity judgement that varies between observers and between days;
- no structured Observation 1 to 3 workflow for the same plant or case;
- the risk of invalid images (non-leaf, non-fruit) entering the monitoring history;
- the absence of a transparent relative severity percentage that can be communicated to the farmer; and
- limited linkage between monitoring results, weather context, and an overall case summary.

### 1.4 Research gap

Tomato disease identification applications are increasingly available, but farmers still lack a focused component that tracks relative severity across repeated observations of the same monitoring case. Existing systems commonly stop at a disease label and do not provide structured recovery trends, visual consistency checks, or Initial, Peak and Final severity summaries. The gap is therefore a unified monitoring solution that combines hybrid image validation, transparent affected-area severity, embedding-based consistency evidence across Observation 1 to 3, observation-based trend states, optional weather context, and a mobile-friendly interface, as summarised in Table 2.1 (Section 2.5).

### 1.5 Aim and objectives

**Aim.** To develop an observation-based tomato disease severity and recovery monitoring component that validates leaf and fruit images, estimates relative severity, evaluates visual consistency across repeated uploads, and presents farmer-facing trend and overall monitoring summaries within the TomatoDoc platform.

**Specific objectives.**

- To design and implement a hybrid image gate using OpenCV pre-validation and MobileNetV2 for leaf and fruit uploads.
- To estimate relative severity primarily from OpenCV affected-area percentage with a defined LOW/HIGH threshold.
- To integrate EfficientNet-B0 for visual embeddings and supporting severity evidence.
- To implement cosine-similarity-based consistency checks between successive accepted observations.
- To compute observation-based trend states and overall Initial/Peak/Final monitoring summaries.
- To attach optional weather context using location (GPS, manual selection, or default).
- To develop a mobile monitoring workflow (create case, upload Observation 1 to 3, result, overall report) using Expo/React Native and FastAPI.
- To evaluate the pipeline through automated tests and reported model and gate metrics.

#### **Business objectives.**

- To help farmers track disease progression with clearer visual and percentage-based feedback.
- To reduce reliance on memory-based monitoring across multiple days.
- To support sustainable crop management by enabling earlier recognition of worsening trends.
- To contribute a specialised monitoring module that complements disease identification within a digital agriculture platform.

### **1.6 Research questions or hypotheses**

- RQ1: Can a hybrid OpenCV and MobileNetV2 gate keep non-leaf and non-fruit images out of the monitoring history without rejecting valid leaf images?
- RQ2: Is OpenCV affected-area percentage with a 30% threshold a usable and explainable relative severity signal for farmers?
- RQ3: Can cosine similarity between EfficientNet-B0 embeddings provide useful evidence that successive uploads belong to the same monitoring sequence?
- RQ4: Can severity scores from repeated observations be converted into clear trend states (BASELINE, IMPROVING, WORSENING, STABLE, RECOVERED)?

### **1.7 Scope limitations and assumptions**

- The component monitors relative severity only. It does not identify diseases and does not prescribe treatments or medicines.

- Disease labels such as `early_blight` or `anthracnose` are supplied by the external identification module or a default for the case.
- Results are relative monitoring aids and are not clinical plant pathology reports.
- A monitoring case contains up to three observations (Observation 1 to 3) of the same plant, leaf, or fruit.
- Weather is used as supporting context and never decides disease identity or severity class.
- The farmer is assumed to have a smartphone with a camera and, for live use, network access to the backend.

## **1.8 Individual contribution**

The individual contribution of this thesis is the design, implementation, and evaluation of the monitoring pipeline described in this report: the hybrid image gate, the OpenCV affected-area severity estimator, the EfficientNet-B0 embedding and consistency module, the trend and overall summary logic, the weather attachment, the FastAPI backend with SQLite storage, the Expo React Native monitoring screens, and the automated test suites.

## **1.9 Thesis structure**

Chapter 2 reviews related work and states the gap. Chapter 3 describes the research methodology, data, procedure, and metrics. Chapter 4 presents requirements and the solution design. Chapter 5 covers implementation, integration, and testing. Chapter 6 reports results and validation. Chapter 7 discusses the findings and limitations. Chapter 8 covers commercialisation and sustainability, and Chapter 9 concludes the thesis and outlines future directions.

## 2 LITERATURE REVIEW AND RELATED WORK

### 2.1 Review method and scope

The review focused on four areas that correspond to the building blocks of the component: convolutional neural networks for plant disease recognition, image-processing approaches to severity estimation, deep embedding similarity for comparing images over time, and the use of weather as contextual information in agriculture. Journal articles, conference papers, dataset papers, and official library documentation were consulted. The aim was to identify what is already established and where repeated-observation severity monitoring is still weakly supported.

### 2.2 Domain and theoretical foundations

**Convolutional neural networks.** CNNs learn hierarchical visual features from images and are the standard approach for image-based plant disease recognition [1], [2]. MobileNetV2 uses inverted residual blocks with linear bottlenecks to keep computation low, which makes it suitable for validation tasks and edge-friendly inference [3]. EfficientNet-B0 is the baseline of a family of models scaled with a compound coefficient and produces a 1280-dimensional feature vector before its classifier, which can be reused as an image embedding [4].

**Colour-space severity estimation.** Severity can be expressed as the share of the subject area that shows symptoms. Colour segmentation, commonly in HSV space, separates healthy tissue from symptomatic regions without needing large labelled datasets, although it can be sensitive to lighting and background [5].

**Embedding similarity.** Two embeddings  $a$  and  $b$  can be compared with cosine similarity,  $\cos(a, b) = (a \cdot b) / (\|a\| \|b\|)$ . A value close to 1 means the images are visually very similar. In monitoring, this is evidence of visual consistency and not proof of plant identity.

### 2.3 Related research

Mohanty et al. showed that deep networks trained on the PlantVillage image collection can recognise many crop diseases with very high accuracy, but also noted that performance falls when test images come from different conditions than the training images [1]. Ferentinos reported high accuracy for CNN-based diagnosis across many plant and disease classes using a large public image database [2]. These studies focus on one-shot classification.

Barbedo described a digital image processing method that detects and measures leaf disease symptoms, expressing severity as the percentage of affected area [5]. This is the idea followed by the primary severity signal in this component. Hughes and Salathé describe the open PlantVillage repository that is widely used as a starting point for tomato leaf work [6].

Weather-aware agriculture systems treat temperature, humidity, and rainfall as contextual risk factors rather than as direct disease classifiers, which is the role weather plays in this component.

## 2.4 Existing systems products and techniques

Most available plant disease applications and research prototypes accept a single photo and return a disease label with a confidence score. Some report a severity grade, but the grade often comes from an opaque classifier output and is not tracked across visits. Few provide an end-to-end farmer workflow that stores repeated observations of the same case, checks that new photos are consistent with earlier ones, and presents an explicit trend. Treatment advice is frequently mixed into the same result, which blurs the boundary between diagnosis, monitoring, and prescription.

## 2.5 Gap synthesis and conceptual framework

*Table 2.1: Research gap summary*

| Aspect      | Existing systems                      | This component                                        |
|-------------|---------------------------------------|-------------------------------------------------------|
| Focus       | Often one-shot disease identification | Repeated severity monitoring                          |
| Severity    | Class label or opaque score           | OpenCV affected-area % with LOW/HIGH                  |
| Consistency | Usually absent                        | EfficientNet embedding cosine similarity              |
| Trend       | Limited or none                       | BASELINE / IMPROVING / WORSENING / STABLE / RECOVERED |
| Weather     | Separate applications                 | Optional contextual weather on each observation       |
| Treatment   | Often mixed with diagnosis            | Monitoring guidance only; no medicine recommendation  |

The conceptual framework of the component is a pipeline. An uploaded image first passes a validation gate. Accepted images receive a transparent affected-area severity. An embedding is compared with the previous accepted observation to give consistency evidence. The severity scores of successive observations are then converted into a trend, and optional weather is attached before everything is stored and summarised for the farmer.

## 3 RESEARCH METHODOLOGY

### 3.1 Research design

The work follows an applied, design-and-build research approach. A working monitoring system was developed in iterations and evaluated with holdout metrics for the models and automated and manual tests for the pipeline. The component is embedded in the TomatoDoc platform and communicates with a FastAPI backend dedicated to monitoring cases and observations.

#### 3.1.1 Agile principles applied

Agile methodology was applied so that the component could be built in small increments. Work was divided into sprints covering gate validation, severity estimation, consistency checks, weather integration, mobile screens, and overall reporting. Automated backend tests and farmer-flow demonstration checks were run after each increment. Risks that were managed included invalid image uploads, model path configuration, weather API dependency, and location permission issues on web and mobile. Regular coordination with the other TomatoDoc members kept API contracts and module boundaries stable. Figure 3.1 shows the iterative cycle that was followed.



Figure 3.1: Agile development lifecycle used for the component

#### 3.1.2 Feasibility and planning

Technical feasibility was established with available PyTorch models (MobileNetV2 gate and EfficientNet-B0), OpenCV for classical area estimation, FastAPI for REST services, SQLite for storage, and Expo React Native for the farmer interface. Operationally, the farmer flow was kept to a simple Observation 1 to 3 sequence with optional location and weather. Economically, the component is software-centric and needs no specialised sensors. Figure 3.2 gives the indicative schedule and Figure 3.3 the work breakdown.

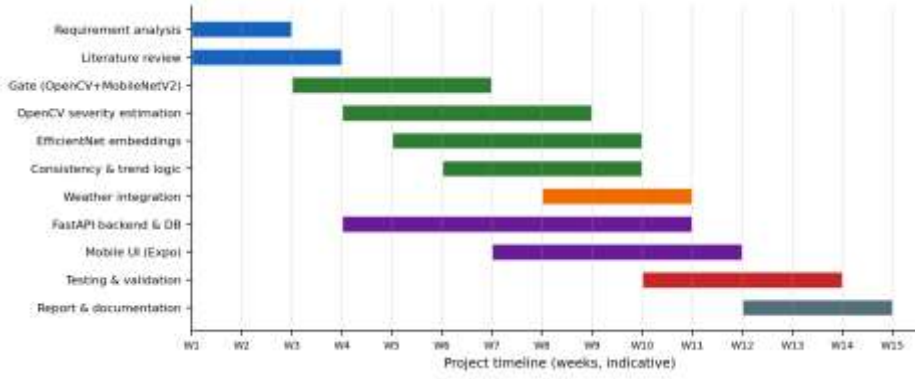


Figure 3.2: Gantt chart for component milestones

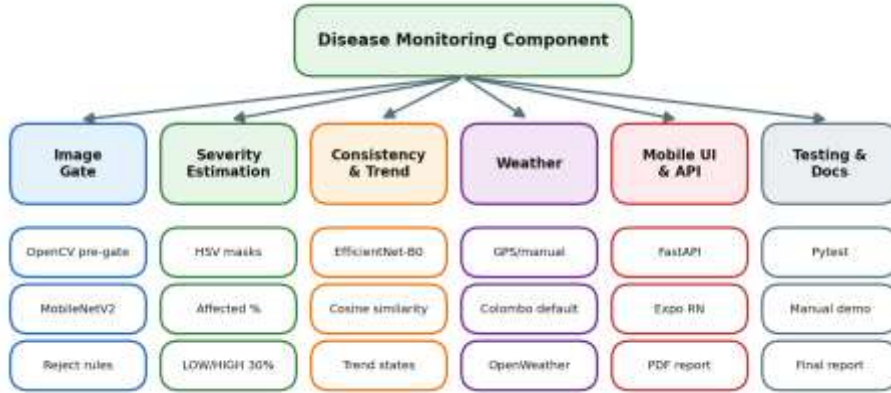


Figure 3.3: Work breakdown structure of the Disease Monitoring component

### 3.2 Data participants or study materials

Gate training and evaluation used PlantVillage tomato leaf images [6] against REJECT samples. The supporting severity CNN was trained with relative pseudo-severity labels derived from YOLO affected-area information (P40). The reported class counts are LOW 2,765 and HIGH 4,141, for a total of 6,906 images (Figure 3.4). Runtime farmer uploads are stored per monitoring case, and calibration folders exist for leaf and fruit reference annotation workflows. No human participants were recruited for the studies reported here. Table 3.1 summarises the study materials.

Table 3.1: Datasets and study materials

| Dataset                                                       | Purpose                                 | Size / content                                        |
|---------------------------------------------------------------|-----------------------------------------|-------------------------------------------------------|
| PlantVillage tomato leaf images and REJECT samples            | Gate training and holdout evaluation    | Holdout: 500 valid leaf images and 225 REJECT samples |
| Pseudo-severity dataset (YOLO affected-area information, P40) | Training of the supporting severity CNN | 6,906 images: LOW 2,765 and HIGH 4,141                |

| Dataset             | Purpose                             | Size / content                                  |
|---------------------|-------------------------------------|-------------------------------------------------|
| Farmer uploads      | Runtime monitoring                  | Stored per monitoring case (Observation 1 to 3) |
| Calibration folders | Leaf and fruit reference annotation | Reference images for annotation workflows       |

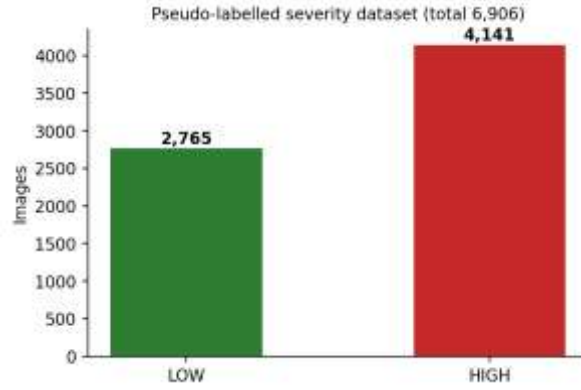


Figure 3.4: Class distribution of the pseudo-labelled severity dataset

### 3.3 Experimental or study procedure

The monitoring procedure begins when a farmer creates a LEAF or FRUIT case and uploads Observation 1. The image is validated by the hybrid gate. If it is accepted, relative severity is estimated, an embedding is stored, and the observation is saved as BASELINE. Observations 2 and 3 repeat the process with a consistency check against the previous accepted observation. Weather context may be attached from the case location. After three observations an overall summary presents Initial, Peak and Final severity, a timeline, and the trend.

#### 3.3.1 Data preprocessing

- Image decoding and conversion to RGB.
- Resizing to  $224 \times 224$  and ImageNet normalisation for MobileNetV2 and EfficientNet-B0.
- OpenCV [7] colour-space conversion (HSV and BGR) for subject and disease masking.
- Optional JPEG downscaling before secondary verification calls.
- Cosine similarity on embeddings (L2-aware) for consistency scoring.

#### 3.3.2 Model training and integration

The leaf gate MobileNetV2 was retrained for tomato-leaf versus REJECT discrimination and saved as gate\_leaf.pth, with a backup copy. The fruit gate uses gate\_fruit.pth. EfficientNet-B0 checkpoints (best\_cnn\_severity\_model.pth and fruit\_severity\_cnn.pth) are loaded for embeddings and for a supporting cnn\_high\_prob value. In the deployed pipeline the farmer-facing severity

percentage comes from OpenCV affected-area estimation, and the CNN does not overwrite it. Table 3.2 lists the models and their roles.

*Table 3.2: Models used in the component*

| Model                   | Role                                                     | Input                | Model file                  |
|-------------------------|----------------------------------------------------------|----------------------|-----------------------------|
| OpenCV pre-gate         | Deterministic rejection of non-leaf and non-fruit images | Decoded image        | Rule-based (no weights)     |
| MobileNetV2 (leaf)      | Gate probability for leaf uploads                        | $224 \times 224$ RGB | gate_leaf.pth               |
| MobileNetV2 (fruit)     | Gate probability for fruit uploads                       | $224 \times 224$ RGB | gate_fruit.pth              |
| EfficientNet-B0 (leaf)  | 1280-D embedding and supporting cnn_high_prob            | $224 \times 224$ RGB | best_cnn_severity_model.pth |
| EfficientNet-B0 (fruit) | 1280-D embedding and supporting cnn_high_prob            | $224 \times 224$ RGB | fruit_severity_cnn.pth      |
| OpenCV severity         | Primary affected-area percentage                         | Accepted image       | Rule-based (no weights)     |

### 3.4 Measures and evaluation metrics

- Gate: leaf pass rate and reject false-accept rate on a holdout set.
- Supporting CNN: test accuracy, test F1, and best validation accuracy.
- Severity: affected-area percentage = affected pixels / subject pixels  $\times 100$ ; class HIGH if the score is above 0.30, otherwise LOW.
- Consistency: cosine similarity thresholds, MATCH at 0.85 or above, POSSIBLE\_MATCH at 0.65 or above, otherwise MISMATCH.
- Trend: comparison of current and previous scores with a stability epsilon of 0.05.
- Pipeline: pass or fail outcome of the automated and manual test cases.

*Table 3.3: Consistency thresholds*

| Status         | Similarity condition                 |
|----------------|--------------------------------------|
| MATCH          | similarity $\geq 0.85$               |
| POSSIBLE_MATCH | $0.65 \leq \text{similarity} < 0.85$ |
| MISMATCH       | similarity $< 0.65$                  |

### 3.5 Data analysis

Model results are reported on held-out data. Test accuracy and test F1 are used for the supporting CNN, and the training accuracy of 98.79% is deliberately not cited as performance because it reflects fitting to the training data. Because the severity labels are pseudo-labels derived from detector output, the CNN metrics are interpreted as agreement with those labels and not as agreement with expert-assessed severity. Pipeline behaviour is analysed through pass or fail test cases that check each stage and the response fields returned to the farmer.

### 3.6 Ethics privacy security and responsible AI

- The images are of crop leaves and fruit and do not contain personal data. Location is optional and, when not provided, a Colombo default is used.
- Farmer-facing API responses hide raw embeddings and secondary verification details, and farmer-facing text avoids exposing internal model or provider names.
- Where an optional external vision API is used for secondary checks, a downscaled JPEG is sent and a fallback is available when the service is not reachable.
- Results are presented as relative monitoring aids. The system makes no clinical diagnosis claim and gives no medicine prescription.
- On a MISMATCH the farmer must confirm before the observation is accepted, so the system does not silently merge photos of different plants.

### 3.7 Reproducibility and research artefacts

The artefacts needed to reproduce the work are the model files (gate\_leaf.pth, gate\_fruit.pth, best\_cnn\_severity\_model.pth, fruit\_severity\_cnn.pth), the FastAPI backend and its observation service, the Expo React Native frontend, the configured thresholds (0.30 severity, 0.85 and 0.65 consistency, 0.05 stability epsilon), and the Pytest automated suites. The REST endpoints used by the mobile client are listed in Section 5.2, and the internal component documentation [13] records the severity methodology and validation reports.

## 4 PRODUCT AND SOLUTION DESIGN

### 4.1 Requirements

#### 4.1.1 User requirements

- Create a leaf or fruit monitoring case.
- Upload repeated observations with clear Observation 1 to 3 guidance.
- Receive a rejection message for invalid images.
- View relative severity percentage and LOW/HIGH class.
- See consistency status and confirm a mismatch when needed.
- View the trend and the overall Initial/Peak/Final summary.
- Optionally attach location and weather for environmental context.
- Download or view a monitoring summary or report.

#### 4.1.2 Functional requirements

- The gate must reject non-leaf and non-fruit images before severity inference.
- The system must compute affected-area percentage and classify LOW/HIGH with a 30% threshold.
- The system must compute embedding similarity against the previous accepted observation.
- The system must derive trend states and an overall summary for each case.
- The system must expose cases, observations, status, insight and PDF report through REST endpoints.

#### 4.1.3 Non-functional requirements

- Usability: a simple Observation 1 to 3 flow suitable for non-technical farmers.
- Transparency: the farmer-facing severity is an explainable percentage.
- Reliability: invalid images must never enter the monitoring history.
- Information hiding: raw embeddings and secondary evidence are not exposed to farmers.

#### 4.1.4 Use case and user stories

Table 4.1 gives the monitoring flow step by step and Table 4.2 describes the main use case scenario. The user stories that follow summarise the needs of the farmer.

*Table 4.1: Use case: tomato farmer monitoring flow*

| Step | Actor  | Action                 | System response   |
|------|--------|------------------------|-------------------|
| 1    | Farmer | Create LEAF/FRUIT case | Case ID generated |

| Step | Actor  | Action                    | System response                       |
|------|--------|---------------------------|---------------------------------------|
| 2    | Farmer | Set location (optional)   | GPS, manual or Colombo default stored |
| 3    | Farmer | Upload Observation 1      | Gate, severity and BASELINE saved     |
| 4    | Farmer | Upload Observation 2 or 3 | Consistency and trend updated         |
| 5    | Farmer | View overall result       | Summary, timeline, insight and PDF    |

*Table 4.2: Use case scenario UC-01*

|                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use case ID</b>           | UC-01                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>Use case name</b>         | Tomato disease severity and recovery monitoring (Observation 1 to 3)                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>Preconditions</b>         | <ol style="list-style-type: none"> <li>1. The mobile application is installed.</li> <li>2. The FastAPI backend and model files are available.</li> <li>3. The device has a camera or photo gallery.</li> <li>4. Location permission is optional.</li> </ol>                                                                                                                                                                                                          |
| <b>Primary actor</b>         | Tomato farmer                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Main success scenario</b> | <ol style="list-style-type: none"> <li>1. The farmer creates a LEAF or FRUIT case.</li> <li>2. The farmer sets the location.</li> <li>3. The farmer uploads Observation 1 and the gate validates it.</li> <li>4. The system estimates severity, stores the embedding and saves BASELINE.</li> <li>5. The farmer uploads Observation 2 and 3; consistency and trend are updated.</li> <li>6. The farmer views the overall summary, insight and PDF report.</li> </ol> |
| <b>Alternative flows</b>     | <p>A1. Invalid image: rejected and not saved.</p> <p>A2. MISMATCH: the farmer must confirm.</p> <p>A3. GPS unavailable: Colombo default keeps weather available.</p>                                                                                                                                                                                                                                                                                                 |
| <b>Postconditions</b>        | Observation history, trend and summary are stored for the case.                                                                                                                                                                                                                                                                                                                                                                                                      |

- As a farmer, I want invalid photos rejected so that the severity history stays meaningful.
- As a farmer, I want severity shown as a percentage so that I can compare days easily.
- As a farmer, I want to know if the new photo looks different from the previous one.
- As a farmer, I want an overall trend after three observations.
- As a farmer, I want optional weather context linked to my location.

## 4.2 Component architecture within the complete solution

The system design separates the farmer interface, the REST API, the model inference utilities, and persistence. The architecture has eight parts: the Expo React Native monitoring screens, the FastAPI observation routers and services, the OpenCV pre-gate and MobileNetV2 gate, the OpenCV affected-area estimator, the EfficientNet-B0 embedding path, the consistency and trend modules, the weather service, and the SQLite observation store. The backend orchestrates gate, severity, consistency, trend, weather, and storage in that order (Figure 4.1).

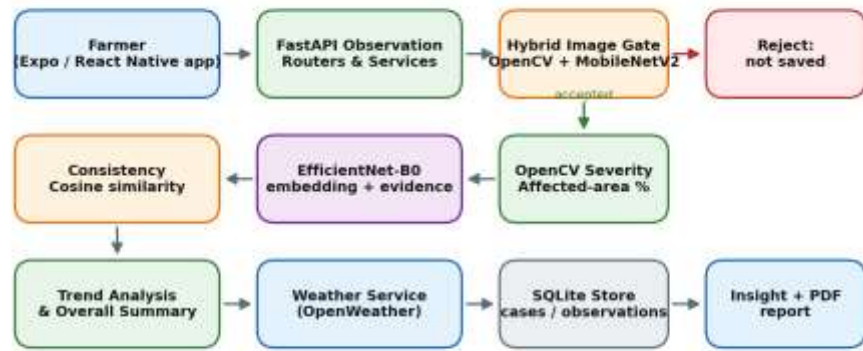


Figure 4.1: System architecture of observation-based monitoring

## 4.3 Design alternatives and decisions

Table 4.3 records the main design decisions, the alternatives that were considered, and the reason for each choice.

Table 4.3: Main design decisions

| Decision         | Alternative considered      | Choice and reason                                                                                |
|------------------|-----------------------------|--------------------------------------------------------------------------------------------------|
| Primary severity | CNN class probability       | OpenCV affected-area %. It is transparent and explainable; the CNN stays as supporting evidence. |
| Image gate       | CNN-only or OpenCV-only     | Hybrid. OpenCV rejects obvious non-leaf content and MobileNetV2 confirms ambiguous cases.        |
| Same-plant check | No check, or manual         | EfficientNet embedding cosine similarity with farmer confirmation on mismatch.                   |
| Storage          | Server database             | SQLite. It is lightweight and sufficient for a demonstrator.                                     |
| Weather          | Use weather to set severity | Context only, so weather cannot change severity class or disease identity.                       |

| Decision         | Alternative considered      | Choice and reason                                            |
|------------------|-----------------------------|--------------------------------------------------------------|
| Disease identity | Identify inside this module | External contract, to keep the module focused on monitoring. |

## 4.4 Detailed design

### 4.4.1 Hybrid image gate

The OpenCV pre-gate performs deterministic rejection of non-leaf and non-fruit content using classical signals (sanity, colour, shape and texture, and modality scoring). MobileNetV2 then provides a probability score. In hybrid mode the CV stage must pass, and a low CV confidence requires MobileNet confirmation. Rejected images never reach severity inference (Figure 4.2).

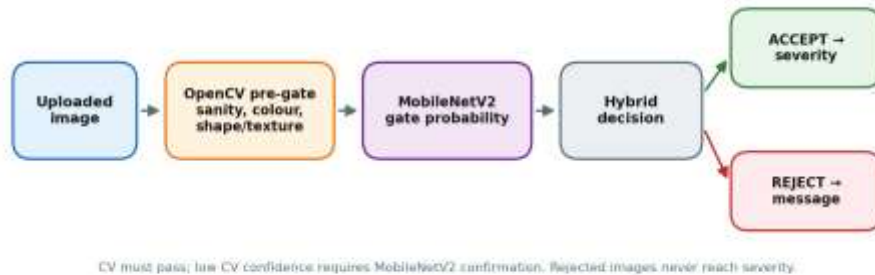


Figure 4.2: Hybrid image gate pipeline (OpenCV and MobileNetV2)

### 4.4.2 Primary severity from OpenCV affected area

The subject region (leaf or fruit) is segmented and diseased pixels are estimated. The affected-area percentage is affected pixels divided by subject pixels multiplied by 100. The severity class is HIGH when the score is above 0.30 and LOW otherwise. This percentage is the farmer-facing relative severity signal (Figure 4.3). The colour segmentation uses standard OpenCV operations [7], [12].

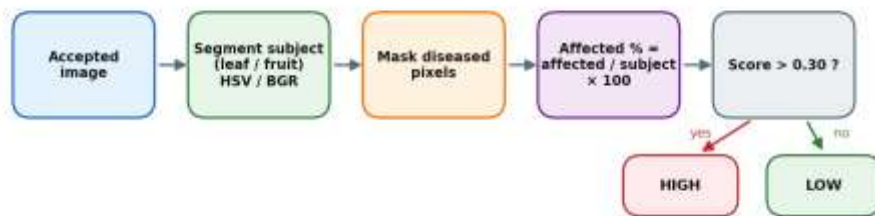


Figure 4.3: OpenCV affected-area severity estimation

### 4.4.3 Embedding and visual consistency

EfficientNet-B0 extracts a 1280-dimensional embedding that is compared by cosine similarity with the embedding of the previous accepted observation. The thresholds are given in Table 3.3. This gives visual consistency evidence and not biometric plant identity. The model also outputs `cnn_high_prob` as supporting backend evidence (Figure 4.4).

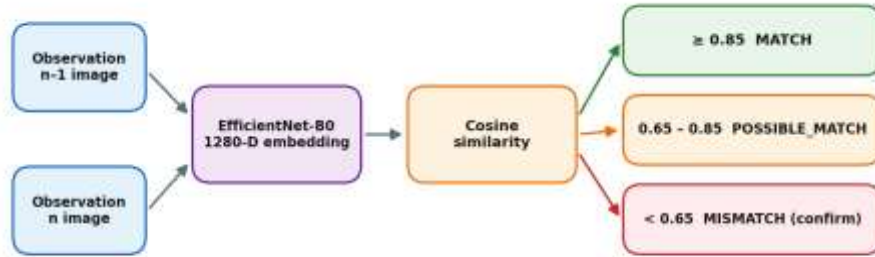


Figure 4.4: EfficientNet-B0 embedding extraction for visual consistency

### 4.4.4 Trend analysis and overall summary

The trend compares the current and previous severity scores. BASELINE is assigned to the first observation. IMPROVING, WORSENING and STABLE use a stability epsilon of 0.05. RECOVERED is assigned when the current score is low and improved relative to the previous observation. The overall summary aggregates the Initial, Peak and Final percentages over the accepted observations (Figure 4.5).

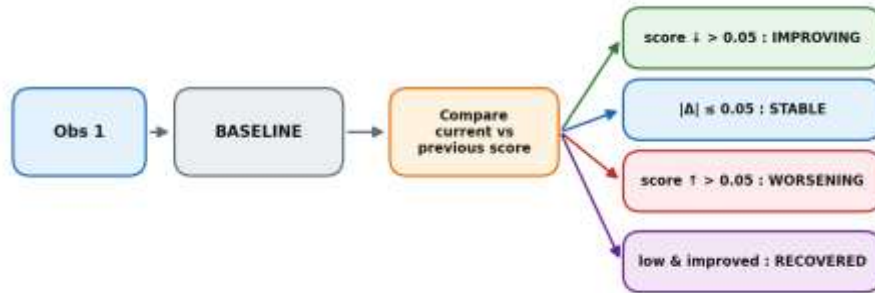


Figure 4.5: Trend analysis decision logic

### 4.4.5 Weather context

When a location is available (GPS, manual selection, or the Colombo default) the backend attaches weather context to the observation using a weather service. The weather values are stored with the observation as supporting information only. The weather service used is OpenWeather [11].

## 4.5 Standards and professional practice

- Communication follows REST over HTTP with JSON payloads.

- Image preprocessing follows the standard ImageNet normalisation expected by the pretrained backbones.
- Sources are cited in IEEE style.
- The component is documented and tested with Pytest suites and keeps clear disclaimers that results are monitoring aids and not clinical reports.

## 5 IMPLEMENTATION INTEGRATION AND TESTING

### 5.1 Development environment and technology stack

Table 5.1 lists the tools used. The backend is written in Python with FastAPI [9] and PyTorch [8], image processing uses OpenCV [7], and the mobile client is built with Expo and React Native [10]. Weather context is obtained from the OpenWeather API [11].

Table 5.1: Development environment and tools

| Layer                 | Tools and technologies                                          |
|-----------------------|-----------------------------------------------------------------|
| Backend               | Python, FastAPI, PyTorch, OpenCV, SQLite, Uvicorn               |
| Frontend              | Expo, React Native, TypeScript                                  |
| Models                | MobileNetV2 gate, EfficientNet-B0                               |
| APIs                  | OpenWeather (weather); optional vision API for secondary checks |
| Testing               | Pytest automated suites                                         |
| IDE and documentation | VS Code / Cursor, Markdown documentation                        |

### 5.2 Component implementation

The backend exposes endpoints such as POST /cases, POST /cases/{id}/observations, GET /cases/{id}/status, GET /cases/{id}/insight, and GET /cases/{id}/report.pdf. The observation service orchestrates the full pipeline described in Chapter 4. The frontend provides the Create Case, Observation Upload, Result, and Overall Monitoring screens with location access, mismatch confirmation, a severity timeline, and summary cards (Figure 5.1).

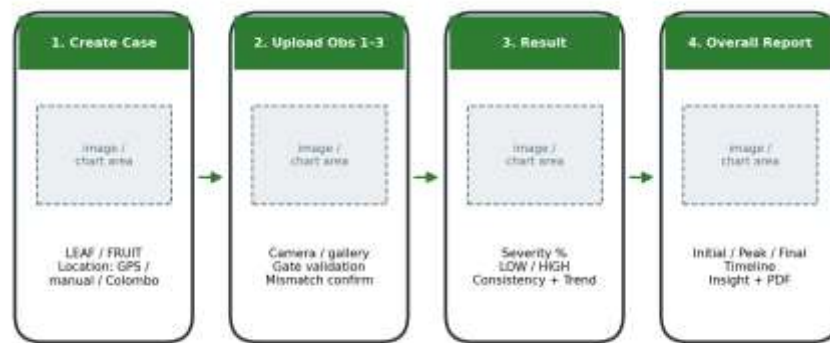


Figure 5.1: Mobile observation workflow (Create Case, Upload, Result, Overall Report)

## 5.3 Integration with the other components

The component integrates with the rest of TomatoDoc through a disease label contract. The label (for example `early_blight` or `anthracnose`) is received from the identification module or set as a default for the case, and it is stored with the monitoring case. No other module is needed to produce severity, consistency or trend, so the component can be developed and tested independently. Treatment and nutrient recommendations are outside this module and are not returned by its API.

## 5.4 Verification and software testing

Automated tests cover observation creation, gate rejection, severity response fields, consistency behaviour, trend and recommendation triggers, weather attachment, monitoring summary generation, and isolation of secondary evidence from farmer API responses. Manual demonstration checks cover the leaf and fruit upload flows in the mobile interface. Table 5.2 summarises the test cases and the detailed test case tables are given in Appendix A.

*Table 5.2: Test case summary for the observation pipeline*

| Test | Scenario                      | Expected result                      |
|------|-------------------------------|--------------------------------------|
| T1   | Valid leaf Observation 1      | Accepted, BASELINE saved             |
| T2   | Same-plant-like Observation 2 | MATCH or POSSIBLE_MATCH and trend    |
| T3   | Visually different image      | MISMATCH, confirmation required      |
| T4   | Invalid or REJECT image       | Gate rejection, not saved            |
| T5   | Worsening sequence            | WORSENING status and guidance path   |
| T6   | Improving or stable sequence  | IMPROVING or STABLE                  |
| T7   | Weather with location         | Weather context attached             |
| T8   | Case status endpoint          | Summary payload for the frontend     |
| T9   | Fruit without model path      | HTTP 503 when unconfigured           |
| T10  | Public observation API        | Evidence and embedding fields hidden |

## 5.5 Deployment and operation

The backend can run on a local machine or LAN server for demonstration, or on a cloud virtual machine. Model file paths are configured on the server. When the fruit model path is not configured, the fruit endpoint returns HTTP 503 instead of producing a result. The mobile application communicates with the backend through its REST API.

## 6 RESULTS AND VALIDATION

### 6.1 Research results

#### 6.1.1 Image gate results

In the evaluated holdout setting the hybrid gate separated valid tomato leaf images from invalid uploads. The leaf gate accepted all 500 valid leaf images (500/500) and made no false accepts on the 225 REJECT samples (0/225), as shown in Figure 6.1. In the application, rejected images are blocked before severity inference. The values are listed in Table 6.1.

Table 6.1: Leaf gate holdout summary

| Metric              | Value                   |
|---------------------|-------------------------|
| Leaf pass           | 500/500                 |
| Reject false-accept | 0/225                   |
| Model file          | ml/models/gate_leaf.pth |

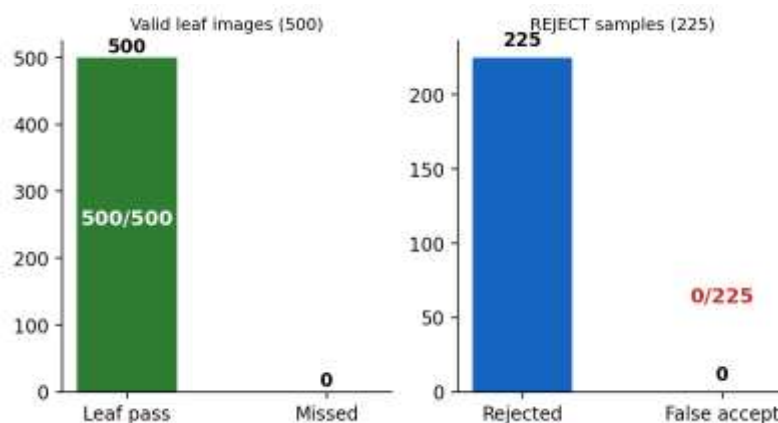


Figure 6.1: Leaf gate holdout results

#### 6.1.2 Supporting severity model results

The leaf EfficientNet-B0 supporting model reached 81.42% test accuracy and 81.60% test F1 on the pseudo-labelled test set, with a best validation accuracy of 86.02% (Table 6.2, Figure 6.2).

Table 6.2: Leaf EfficientNet supporting model metrics

| Metric                   | Value  |
|--------------------------|--------|
| Test accuracy            | 81.42% |
| Test F1                  | 81.60% |
| Best validation accuracy | 86.02% |

| Metric | Value                                               |
|--------|-----------------------------------------------------|
| Note   | Train accuracy (98.79%) is not cited as performance |

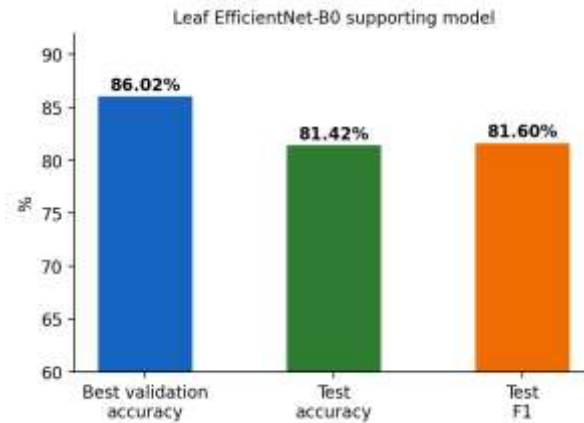


Figure 6.2: Leaf EfficientNet-B0 supporting model metrics

## 6.2 Product and technical results

Relative severity is shown to the farmer as an affected-area percentage with a LOW/HIGH class using the 30% threshold. Across Observation 1 to 3 the system produces a severity timeline and an overall Initial, Peak and Final summary, whose structure is shown in Figure 6.3. Cosine similarity on the embeddings gives MATCH, POSSIBLE\_MATCH or MISMATCH evidence (Figure 6.4), and the trend labels BASELINE, IMPROVING, WORSENING, STABLE and RECOVERED are produced from changes in the severity score.

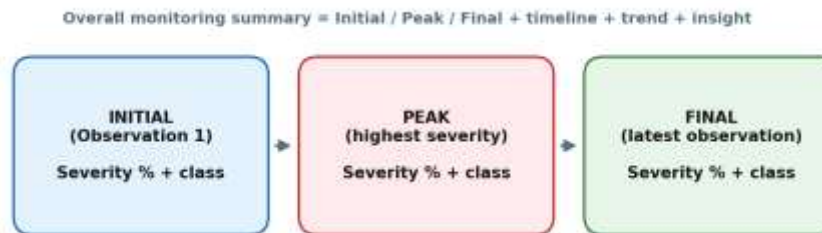


Figure 6.3: Structure of the overall monitoring summary (Initial / Peak / Final)

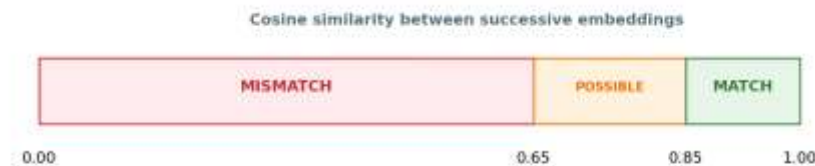


Figure 6.4: Visual consistency thresholds used across Observation 1 to 3

## 6.3 User or stakeholder validation

A formal study with farmers was not carried out for this component. Validation to date consists of manual demonstration runs of the leaf and fruit flows in the mobile interface, covering case creation, location selection, upload, result review, mismatch confirmation, and the overall report. During these runs the farmer-facing text did not expose internal model or provider names, and the Colombo fallback kept weather available when GPS was not. A structured usability study with farmers is listed as future work.

## 6.4 Integration results

The disease label contract allowed the monitoring component to run without the other TomatoDoc modules. The automated test T10 confirmed that embeddings and secondary evidence are not present in the public observation API responses, and T9 confirmed that an unconfigured fruit model returns HTTP 503.

## 6.5 Summary of findings

Table 6.3 brings together the main findings of the evaluation.

*Table 6.3: Summary of findings*

| Area           | Finding                                                                          |
|----------------|----------------------------------------------------------------------------------|
| Image gate     | 500/500 valid leaf images passed and 0/225 REJECT samples were falsely accepted. |
| Supporting CNN | 81.42% test accuracy and 81.60% test F1 on pseudo-labelled data.                 |
| Severity       | Transparent affected-area percentage with a 30% LOW/HIGH threshold.              |
| Consistency    | Cosine similarity thresholds give MATCH, POSSIBLE_MATCH and MISMATCH evidence.   |
| Trend          | Five trend states and an Initial/Peak/Final summary over three observations.     |
| Pipeline       | Automated suites cover the stages listed in Section 5.4.                         |

## **7 DISCUSSION**

### **7.1 Interpretation of findings**

Using OpenCV as a hard pre-gate reduces the chance of unrelated images being accepted, while MobileNetV2 helps with ambiguous cases in hybrid mode. This favours the reliability of the monitoring history over accepting every upload. The affected-area percentage is easy to explain to a farmer, and keeping the CNN output as supporting evidence avoids relying on a model trained on pseudo-labels for the number that is shown. Embedding similarity is useful evidence that successive photos look alike, but it is not a guarantee of plant identity, so farmer confirmation on a mismatch remains necessary. Trend labels are relative comparisons between observations and are not recovery certificates.

### **7.2 Comparison with literature baselines and existing solutions**

Studies such as [1] and [2] report very high accuracy for disease classification, whereas the supporting model here reaches 81.42% test accuracy. The numbers are not directly comparable. Those studies classify disease types, while this model predicts a LOW or HIGH pseudo-severity label, which is a noisier target. The contribution of this component relative to existing solutions lies in the workflow around the models: repeated observations, consistency evidence, trend states and a transparent severity percentage.

### **7.3 Research contribution**

The work shows that a hybrid of classical computer vision and lightweight and mid-capacity CNNs can support a repeatable observation workflow, where each model is given the role it suits best: the CV stage and MobileNetV2 for validation, OpenCV for the primary severity number, and EfficientNet-B0 for embeddings.

### **7.4 Product contribution**

The product contribution is a working mobile and backend system that lets a farmer create a case, upload up to three observations, and view severity, consistency, trend, weather context, insight and a PDF report, while keeping a clear boundary with disease identification and treatment.

### **7.5 Limitations and threats to validity**

- The severity labels used for the CNN are pseudo-labels derived from detector output, so the CNN metrics measure agreement with those labels and not with expert assessment.
- HSV-based segmentation can be affected by lighting and background conditions.
- The gate was evaluated on a PlantVillage-style holdout, and results on field photographs may differ.
- Metrics for the fruit models are not reported in this thesis.

- The gap between the training accuracy (98.79%) and the test accuracy (81.42%) indicates overfitting to the training data.
- No structured study with farmers has yet been run.
- Visual similarity cannot prove that two photos show the same plant.

## **7.6 Implications**

Farmers can use the component to compare condition across days with a percentage and a trend instead of memory. For the wider platform, the clear separation between identification, monitoring and treatment means each module can be improved independently.

## 8 COMMERCIALISATION AND SUSTAINABILITY

### 8.1 Value proposition and target users

The component gives structured recovery tracking and early warning of worsening trends using only a smartphone. Target users are tomato farmers, farmer groups and cooperatives, and agricultural extension services.

### 8.2 Market and competitive position

Existing plant disease applications are mostly one-shot identification tools. This component competes on repeated-observation monitoring, transparent severity, and consistency checks, and it is designed to complement identification tools rather than replace them.

### 8.3 Business or sustainability model

The component can be offered as part of a tomato advisory mobile platform subscription for farmer groups, cooperatives, or agri-extension services. The software-centric cost items are listed in Table 8.1.

*Table 8.1: Budget plan (indicative)*

| Item                | Purpose                            | Indicative notes                     |
|---------------------|------------------------------------|--------------------------------------|
| Development effort  | Backend, mobile and testing        | Student research project             |
| Compute / hosting   | API inference server               | Local/LAN demo or cloud VM           |
| Weather API         | Environmental context              | Free or paid tier depending on usage |
| Optional vision API | Secondary verification and insight | Usage-based; fallback available      |
| Smartphone          | Farmer client device               | Existing farmer phone                |

### 8.4 Scalability intellectual property and compliance

For wider deployment the SQLite store would be replaced by a server database and inference would be hosted on scalable infrastructure. The open-source libraries and the PlantVillage data used in the project have their own licence terms, which must be checked before any commercial release. Clear disclaimers are required: results are relative monitoring support tools, not clinical plant pathology reports or treatment prescriptions.

### 8.5 Contribution to the Sustainable Development Goals

The component supports SDG 2 (Zero Hunger) by helping farmers notice worsening crop condition earlier, and SDG 9 (Industry, Innovation and Infrastructure) by providing a digital tool

for smallholder agriculture. By giving farmers evidence of whether a plant is recovering, it can also support more considered crop management decisions.

## 9 CONCLUSION AND FUTURE DIRECTIONS

### 9.1 Conclusion

This component implements observation-based tomato leaf and fruit disease severity and recovery monitoring for the TomatoDoc platform. Images are validated by a hybrid OpenCV and MobileNetV2 gate. Relative severity is estimated primarily from OpenCV affected-area percentage, while EfficientNet-B0 provides embeddings for visual consistency evidence and supporting backend signals. Repeated observations produce trend states and overall monitoring summaries, with optional weather context. The system addresses the need for transparent, repeatable monitoring beyond one-shot disease identification, without claiming clinical diagnosis or prescribing treatments.

### 9.2 Achievement of objectives

Table 9.1 compares the outcome with each specific objective set in Section 1.5.

*Table 9.1: Achievement of objectives*

| Objective                                   | Outcome                                                                  |
|---------------------------------------------|--------------------------------------------------------------------------|
| Hybrid image gate                           | Implemented; 500/500 leaf pass and 0/225 false-accept on holdout         |
| OpenCV affected-area severity with LOW/HIGH | Implemented with a 30% threshold                                         |
| EfficientNet-B0 embeddings                  | Implemented; 81.42% test accuracy and 81.60% F1 for the supporting model |
| Cosine-similarity consistency checks        | Implemented with 0.85 and 0.65 thresholds                                |
| Trend states and overall summary            | Implemented (five trend states, Initial/Peak/Final)                      |
| Optional weather context                    | Implemented with GPS, manual and default location                        |
| Mobile monitoring workflow                  | Implemented with Expo React Native and FastAPI                           |
| Evaluation through tests and metrics        | Automated suites and manual demonstration; no farmer study yet           |

### 9.3 Individual research and product contributions

The research contribution is the combination of classical affected-area estimation with embedding-based consistency evidence in a repeated-observation workflow. The product contribution is the working monitoring application and backend described in this thesis.

## 9.4 Future directions

- Collect expert-annotated severity labels to replace pseudo-labels and re-evaluate the models.
- Calibrate the 30% threshold and add finer severity bands with agronomist input.
- Report metrics for the fruit models and test the gate on field photographs.
- Extend the workflow beyond three observations and add revisit reminders.
- Add Sinhala and Tamil language support and voice assistance.
- Support offline on-device inference and provide a web dashboard for agricultural officers.
- Run a structured usability study with farmers.

## REFERENCES

- [1] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, Art. no. 1419, 2016.
- [2] K. P. Ferentinos, "Deep learning models for plant disease detection and diagnosis," *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, 2018.
- [3] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.
- [4] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. Int. Conf. Machine Learning (ICML)*, PMLR vol. 97, 2019, pp. 6105–6114.
- [5] J. G. A. Barbedo, "An automatic method to detect and measure leaf disease symptoms using digital image processing," *Plant Disease*, vol. 98, no. 12, pp. 1709–1716, 2014.
- [6] D. P. Hughes and M. Salathé, "An open access repository of images on plant health to enable the development of mobile disease diagnostics," *arXiv:1511.08060*, 2015.
- [7] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.
- [8] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [9] FastAPI Documentation, "FastAPI framework," [Online]. Available: <https://fastapi.tiangolo.com>
- [10] Expo Documentation, "Expo and React Native," [Online]. Available: <https://docs.expo.dev>
- [11] OpenWeather, "Weather API documentation," [Online]. Available: <https://openweathermap.org/api>
- [12] OpenCV Documentation, "Image processing in OpenCV," [Online]. Available: <https://docs.opencv.org>
- [13] TomatoDoc project internal component documentation: observation monitoring, severity methodology, and validation reports.

## GLOSSARY

| Term                     | Definition                                                                 |
|--------------------------|----------------------------------------------------------------------------|
| Monitoring case          | A record for one plant, leaf or fruit that holds up to three observations. |
| Observation              | One accepted photo in a case (Observation 1, 2 or 3).                      |
| Gate                     | The validation stage that accepts or rejects an uploaded image.            |
| Affected-area percentage | Diseased pixels divided by subject pixels, multiplied by 100.              |

| <b>Term</b>       | <b>Definition</b>                                                                             |
|-------------------|-----------------------------------------------------------------------------------------------|
| Embedding         | A numeric feature vector (1280 values for EfficientNet-B0) that describes an image.           |
| Cosine similarity | A measure of how similar two embeddings are, from 0 to 1 for the values used here.            |
| Pseudo-label      | A label derived automatically from another model or tool instead of from an expert.           |
| <b>BASELINE</b>   | Trend status for the first observation of a case.                                             |
| <b>RECOVERED</b>  | Trend status when the current score is low and improved relative to the previous observation. |

## APPENDICES

### Appendix A Research instruments or extended results

This appendix gives the detailed test case tables referred to in Section 5.4 (Table A.1 to Table A.10).

*Table A.1: Test case TC01*

|                         |                                               |
|-------------------------|-----------------------------------------------|
| <b>Test case ID</b>     | TC01                                          |
| <b>Steps</b>            | Create a LEAF monitoring case                 |
| <b>Description</b>      | Farmer creates a case with optional location. |
| <b>Expected outcome</b> | Case ID generated and stored.                 |
| <b>Actual outcome</b>   | Case created and Case ID returned.            |
| <b>Result</b>           | Pass                                          |

*Table A.2: Test case TC02*

|                         |                                                    |
|-------------------------|----------------------------------------------------|
| <b>Test case ID</b>     | TC02                                               |
| <b>Steps</b>            | Upload valid leaf Observation 1                    |
| <b>Description</b>      | Farmer uploads a valid tomato leaf image.          |
| <b>Expected outcome</b> | Image accepted, severity computed, BASELINE saved. |
| <b>Actual outcome</b>   | Observation accepted and saved as BASELINE.        |
| <b>Result</b>           | Pass                                               |

*Table A.3: Test case TC03*

|                         |                                                    |
|-------------------------|----------------------------------------------------|
| <b>Test case ID</b>     | TC03                                               |
| <b>Steps</b>            | Upload same-plant Observation 2                    |
| <b>Description</b>      | Farmer uploads a visually similar follow-up image. |
| <b>Expected outcome</b> | MATCH or POSSIBLE_MATCH with trend updated.        |
| <b>Actual outcome</b>   | Consistency and trend returned as expected.        |
| <b>Result</b>           | Pass                                               |

*Table A.4: Test case TC04*

|                         |                                                       |
|-------------------------|-------------------------------------------------------|
| <b>Test case ID</b>     | TC04                                                  |
| <b>Steps</b>            | Upload visually different image                       |
| <b>Description</b>      | Image differs strongly from the previous observation. |
| <b>Expected outcome</b> | MISMATCH and farmer confirmation required.            |
| <b>Actual outcome</b>   | Mismatch flagged and confirmation requested.          |
| <b>Result</b>           | Pass                                                  |

*Table A.5: Test case TC05*

|                         |                                               |
|-------------------------|-----------------------------------------------|
| <b>Test case ID</b>     | TC05                                          |
| <b>Steps</b>            | Upload invalid / REJECT image                 |
| <b>Description</b>      | Farmer uploads a non-leaf image.              |
| <b>Expected outcome</b> | Gate rejection; observation not saved.        |
| <b>Actual outcome</b>   | Image rejected and not stored in the history. |
| <b>Result</b>           | Pass                                          |

*Table A.6: Test case TC06*

|                         |                                         |
|-------------------------|-----------------------------------------|
| <b>Test case ID</b>     | TC06                                    |
| <b>Steps</b>            | Worsening sequence                      |
| <b>Description</b>      | Severity increases across observations. |
| <b>Expected outcome</b> | WORSENING status and guidance path.     |
| <b>Actual outcome</b>   | WORSENING trend displayed.              |
| <b>Result</b>           | Pass                                    |

*Table A.7: Test case TC07*

|                         |                                             |
|-------------------------|---------------------------------------------|
| <b>Test case ID</b>     | TC07                                        |
| <b>Steps</b>            | Improving or stable sequence                |
| <b>Description</b>      | Severity decreases or stays within epsilon. |
| <b>Expected outcome</b> | IMPROVING or STABLE trend.                  |

|                       |                               |
|-----------------------|-------------------------------|
| <b>Actual outcome</b> | Correct trend label produced. |
| <b>Result</b>         | Pass                          |

*Table A.8: Test case TC08*

|                         |                                              |
|-------------------------|----------------------------------------------|
| <b>Test case ID</b>     | TC08                                         |
| <b>Steps</b>            | Weather with location                        |
| <b>Description</b>      | Case has GPS, manual or default location.    |
| <b>Expected outcome</b> | Weather context attached to the observation. |
| <b>Actual outcome</b>   | Weather context attached.                    |
| <b>Result</b>           | Pass                                         |

*Table A.9: Test case TC09*

|                         |                                |
|-------------------------|--------------------------------|
| <b>Test case ID</b>     | TC09                           |
| <b>Steps</b>            | Fruit without model path       |
| <b>Description</b>      | Fruit model is not configured. |
| <b>Expected outcome</b> | HTTP 503 returned.             |
| <b>Actual outcome</b>   | HTTP 503 returned as designed. |
| <b>Result</b>           | Pass                           |

*Table A.10: Test case TC10*

|                         |                                          |
|-------------------------|------------------------------------------|
| <b>Test case ID</b>     | TC10                                     |
| <b>Steps</b>            | Public observation API                   |
| <b>Description</b>      | Farmer-facing API response is inspected. |
| <b>Expected outcome</b> | Embedding and evidence fields hidden.    |
| <b>Actual outcome</b>   | Internal evidence fields not exposed.    |
| <b>Result</b>           | Pass                                     |

## Appendix B Academic integrity and AI use disclosure

The student is responsible for completing this appendix in accordance with the module policy. List each tool used, its purpose, the verification performed, and the sections or artefacts affected.

| Tool              | Purpose           | Verification performed | Sections affected |
|-------------------|-------------------|------------------------|-------------------|
| [to be completed] | [to be completed] | [to be completed]      | [to be completed] |